

Lab3实验报告

思考题

Thinking 3.1

获得索引的函数ENVX去掉了envid的前22位，只取了后10位，而前22位才能保证进程的唯一性，因为后10位相同可能只是进程的物理位置与另一进程相同，因此，`e->env_id != envid`这一步确定进程e的id确实是传入的envid。

Thinking 3.2

UTOP时是用户可以使用的最高地址，ULIM是用户空间的最高地址，两者之间的区域用户不能直接修改。

`env_cr3`储存进程页目录的物理地址，赋值完成页目录自映射。

虚拟地址是用户和进程使用的地址，要通过操作系统的映射完成虚拟地址到物理地址之间的转换。

Thinking 3.3

在函数 `load_icode_mapper` 中，被传入的 `user_data` 被用于这样一个语句中：

```
struct Env *env = (struct Env *)user_data;
```

因此 `user_data` 就是进程指针。在调用 `load_elf` 的 `load_icode` 中，发现调用 `load_elf` 时的语句为：

```
r = load_elf(binary, size, &entry_point, e, load_icode_mapper);
```

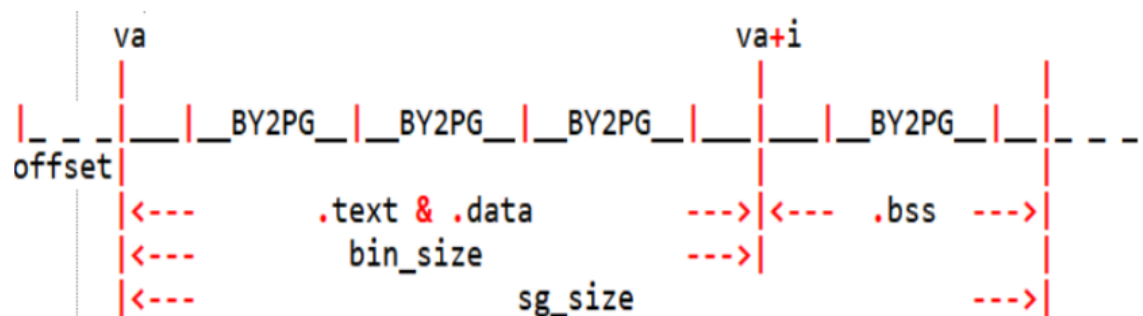
而其中的 `e` 则为传入 `load_icode` 中的 `struct Env *e`

因此没有进程指针，加载镜像显然不能完成。

实例： `stdlib.h` 中的快排函数 `qsort` 中需要传入比较函数 `int(__cdecl compare)(const void, const void*)`

Thinking 3.4

由指导书最糟糕情况的示意图，分析出如下几种特殊情况



`.text & .data:`

第一段的前半段已经装载过内容，无需在这一段 `alloc`，也不能 `insert`。

`offset = 0`，此时从最开始的所有端可以当做正常页处理。

`.test & .data` 与 `.bss` 被某一个页分割恰好切开，不存在共同占用一个 page 的情况。

`.test & .data` 这一段只占用不到一页，因此需要同时对两侧的页面分割进行判定与相应操作。

`.bss`：

前半段和 `.text & .data` 段共用同一页，因此无需 `alloc` 和 `insert`。

与上一种情况相对，即 `.text & .data` 段和 `.bss` 段正好被分割成两页，那么就需要新分配页面。

`.bss` 这一段只占用不到一页，因此需要同时对两侧的页面分割进行判定与相应操作。

Thinking 3.5

虚拟地址

`*entry_point = ehdr->e_entry;` 语句对 `entry_point` 赋值，所以其值对每个进程是一样的，这种统一是由于都是从 ELF 文件中的同部分进行取值。

Thinking 3.6

EPC 是用来存放异常中断发生时进程正在执行的指令的地址的寄存器，将 `env_tf.pc` 设置为 `epc` 就是保存当前进程的上下文信息。

Thinking 3.7

在 `env_destroy` 中，将存于 `KERNEL_SP` 的进程状态复制到 `TIMESTACK` 处。

在发生中断时将进程的状态保存到 `TIMESTACK` 中，在发生系统调用时，将进程的状态保存到 `KERNEL_SP` 中。

Thinking 3.8

```
git@20373324:~/20373324$ grep -r handle_int
Binary file gxemul/vmlinux matches
lib/traps.c:extern void handle_int();
lib/traps.c:    set_except_vector(0, handle_int);
Binary file lib/traps.o matches
lib/genex.S:NESTED(handle_int, TF_SIZE, sp)
lib/genex.S:END(handle_int)
```

`handle_int` 在 `lib/genex.S` 中实现，`handle_sys` 在 `lib/syscall.S` 中实现，其余三个函数只在二进制文件 `lib/traps.o` 和 `lib/genex.o` 中找到。

Thinking 3.9

`set_timer`:

`LEAF(set_timer)`

`li t0, 0xc8`

`sb t0, 0xb5000100` 向 `0xb5000100` 位置写入 `0xc8`，其中 `0xb5000000` 是模拟器 (`gxemul`) 映射实时钟的位置。偏移量为 `0x100` 表示来设置实时钟中断的频率，`0xc8` 则表示 1 秒钟中断 200 次

`sw sp, KERNEL_SP` 将栈指针设为 `KERNEL_SP` 从而能够正确产生时钟中断

```
setup_c0_status STATUS_CU0|0x1001 0    用宏函数setup_c0_status来设置CP0_STATUS的值
jr ra    返回

nop

END(set_timer)
```

```
timer_irq:

sb zero, 0xb5000110

1: j    sched_yield        跳转到调度函数
   nop
   j    ret_from_exception  跳转到中断返回函数
   nop
```

Thinking 3.10

操作系统中设置两个就绪队列，每个进程拥有一个时间片起计时作用，一旦时间片的时间走完，则代表该进程需要执行时钟中断操作，则再将这个进程移动到就绪队列的尾端，并复原其时间片，再让就绪队列最首端的进程执行相应的时间片段。

练习题

Exe 3.2

初始化的链表包括空闲链表env_free_list和两个runnable list: env_sched_list[2]

将envs元素插入空闲链表顺序为逆序，即从envs[NENV-1]开始，为了第一次调用env_alloc()时返回envs[0]

Exe 3.3

如果envid是0，返回当前进程控制块。

获得envid对应的进程索引：ENVX(envid)，然后在envs数组中找到对应的元素给e。

当有checkperm=1时，检查当前进程 curenv 是不是有合法perm去操作这个特定进程（要么e是当前进程本身或e是它的直接子进程，若两者都不是则返回错误。

Exe 3.6

这个函数是个人感觉最难的地方，需要静下心来仔细考虑不同对齐的情下循环的使用，具体情况讨论见Thinking 3.4

Exe 3.7

用户栈是可写的，所以perm设为PTE_V|PTE_R

注意，栈是从高到低增长的，所以最高位的页框，虚拟地址的基地址应该是USTACKTOP-BY2PG

Exe 3.10

lab3-2中最关键的一步，总体思路是：

1. 判断当前时间片是否用完，如果用完则把当前进程移至队尾
2. 判断当前队列是否为空，如果为空则切换至另一队列
3. 如果当前时间片用完，则在队列中寻找可运行的进程，并更新时间片为进程的时间片，注意队列中不只有可运行的进程
4. 时间片倒计时减一，并运行进程

体会与感想

本次实验难度比起lab2有明显增加，在宏观和微观上都有比较大的难度。宏观上的难度在于第一部分需要理解进程结构体的构成，以及理解如何从创建进程到运行进程的整个过程。微观上的难度主要在于第一部分中加载二进制镜像部分，非常考验细节的，需要想清楚每一种对齐情况的处理方法。第二部分任务主要在于理解操作系统处理中断的流程，具体填写代码的难度没有那么难。总的来说我觉得lab3的难度比前几次明显上了一个档次，课下花费的时间也成倍增加，两部分加起来应该花了有25h。